

HTML Forms Assignment

Product review form | Validation | POST and DevTools | Food order form

Name: _____ Roll No.: _____ Date: _____

Note: To try any of the forms, put the code inside the `<body>` of a normal HTML file and open it in your browser.

Question 1: Product review form

Create an HTML form for a Flipkart-style product review with input fields for username, rating (dropdown from 1 to 5) and a textarea for the review message. Use appropriate labels for each field.

Code

```
<h2>Rate this Product</h2>
<form action="#" method="get">

  <label for="username">Username</label>
  <input type="text" id="username" name="username">

  <label for="rating">Rating</label>
  <select id="rating" name="rating">
    <option value="1">1 - Poor</option>
    <option value="2">2 - Fair</option>
    <option value="3">3 - Good</option>
    <option value="4">4 - Very Good</option>
    <option value="5">5 - Excellent</option>
  </select>

  <label for="review">Review</label>
  <textarea id="review" name="review" rows="5" cols="40"></textarea>

  <button type="submit">Submit Review</button>
</form>
```

- Each `<label>` has a `for` value that matches the `id` of its field. This links them, so clicking the label focuses the field and screen readers announce it properly.
- The dropdown (`<select>`) lists ratings 1 to 5 as `<option>` tags.
- Every field has a `name` attribute. Only fields with a `name` are sent when the form is submitted, and Question 3 depends on this.

Question 2: HTML5 validation

Add HTML5 validation so that username is required, rating must be selected, and the review is at least 20 characters. Hint: use `required`, `minlength` and a default unselectable option for the dropdown.

Code

```
<form action="#" method="get">

  <label for="username">Username</label>
  <input type="text" id="username" name="username" required>

  <label for="rating">Rating</label>
  <select id="rating" name="rating" required>
    <option value="" disabled selected>Select a rating</option>
    <option value="1">1 - Poor</option>
    <option value="2">2 - Fair</option>
    <option value="3">3 - Good</option>
    <option value="4">4 - Very Good</option>
    <option value="5">5 - Excellent</option>
  </select>

  <label for="review">Review</label>
  <textarea id="review" name="review" rows="5" cols="40"
    required minlength="20"></textarea>

  <button type="submit">Submit Review</button>
</form>
```

- **Username is required:** `required` stops the form from being submitted while the field is empty. The browser shows a message such as 'Please fill out this field.'
- **Rating must be selected:** the first option, 'Select a rating', has `value=""`, so it counts as no choice. The `disabled selected` attributes make it show first but stop the user from choosing it again after picking a rating. Together with `required` on the `<select>`, the user must pick 1 to 5.
- **Review at least 20 characters:** `minlength="20"` on the textarea, together with `required`, blocks a review that is empty or too short. The browser shows a message such as 'Please lengthen this text to 20 characters or more.'
- The browser checks these rules before sending anything. This is only a convenience for the user, because anyone can bypass it, so the server must check the data again.

Question 3: POST method and the Network tab

Modify the form to use the POST method and, after submitting a test review, open the DevTools Network tab to preview the POST request data. Hint: look for the form submission request and expand the payload/details.

Change only the opening form tag

```
<form action="https://httpbin.org/post" method="post">
```

- `method="post"` sends the values inside the body of the request, not in the web address.
- A form needs somewhere to send the data. `httpbin.org` is a free public testing service that replies with the data it received. Enter only made-up test data, never real personal details, because this is a third-party site. If you have your own test server, use its address instead.

Steps to see the data

1. Open the HTML file in Chrome and press **F12** to open DevTools.

2. Click the **Network** tab, and tick **Preserve log** so the request stays listed after the page changes.
3. Fill in the form with test values, for example username `riya_patel`, rating 4, and a review of 20 or more characters. Click **Submit Review**.
4. In the request list, click the entry named `post`. Its Method shows **POST**.
5. Open the **Payload** tab and look at the form data. You should see:

```
username: riya_patel
rating: 4
review: The phone works well and the battery lasts all day.
```

In the **Headers** tab you can also see Content-Type:

`application/x-www-form-urlencoded`, which is how the form encodes the values.

Why POST matters: with `method="get"` the same values would appear in the address bar (`?username=riya_patel&rating=4...`), where anyone can see them. With POST they travel in the request body, which is the right choice for reviews, passwords and other data that changes something on the server.

Question 4: Zomato-style food order form

Build a food order form with a user name input, a select dropdown for food item (at least 3 options), a quantity (number input) and a textarea for delivery instructions. All fields must be labelled and use required validation.

Code

```
<h2>Place Your Order</h2>
<form action="https://httpbin.org/post" method="post">

  <label for="customer">Your Name</label>
  <input type="text" id="customer" name="customer" required>

  <label for="food">Food Item</label>
  <select id="food" name="food" required>
    <option value="" disabled selected>Choose a dish</option>
    <option value="margherita-pizza">Margherita Pizza</option>
    <option value="paneer-butter-masala">Paneer Butter Masala</option>
    <option value="masala-dosa">Masala Dosa</option>
    <option value="veg-biryani">Veg Biryani</option>
  </select>

  <label for="quantity">Quantity</label>
  <input type="number" id="quantity" name="quantity" min="1" max="10" required>

  <label for="instructions">Delivery Instructions</label>
  <textarea id="instructions" name="instructions" rows="4" cols="40"
    required></textarea>

  <button type="submit">Place Order</button>
</form>
```

- All four fields have a label linked by `for` and `id`, and all four have `required`.
- The dropdown has four dishes, which is more than the 3 required, plus a disabled placeholder so the user must choose one.
- `type="number"` gives a number-only input, and `min="1" max="10"` stops zero, negative or very large quantities.
- The question asks for required validation on every field, so the instructions box is required too. In a real app this box is usually optional, and you would remove `required` from it.

Question 5: Disabled order ID field

Add a disabled input field to the food order form that displays a fixed order ID (e.g. ORD12345) and explain in a comment why this field should be disabled for users.

Code (add inside the form from Question 4, above the button)

```
<!--  
  The order ID is disabled because it is created by the system, not by the user.  
  Disabling it stops the user from editing it, so orders cannot be mixed up or  
  duplicated by mistake. Note: disabled fields are NOT sent when the form is  
  submitted, so the server should create or check the order ID itself.  
-->  
<label for="orderId">Order ID</label>  
<input type="text" id="orderId" name="orderId" value="ORD12345" disabled>
```

Why it should be disabled

- The order ID identifies the order for payment, tracking and support. If users could change it, they could enter another customer's ID or a duplicate one.
- A disabled field is greyed out and cannot be edited or focused, which tells the user it is fixed information for viewing only.
- **Disabled and readonly differ:** a disabled field is not submitted with the form, while a readonly field is still sent. Disabling is the safer choice when the ID is only displayed and the server makes it.
- Even a disabled or hidden field can be edited by someone using DevTools, as you saw in an earlier assignment. So the real protection is on the server, which must generate the order ID and ignore any ID sent from the browser.